

L4a: Graph and Tree Representations

CHEME 5800 Cornell University

Objectives for Today

Today we describe graph structure, calculate graph measures, and choose a storage form that matches the operation an algorithm performs.

Today's concepts

- **Graph language and measures**: define walks, paths, cycles, connectivity, degree, and density.
- **Graph families**: recognize complete graphs, bipartite graphs, and trees from their defining properties.
- **Storage representations**: compare edge lists, adjacency matrices, and adjacency lists by storage and access cost.

Lecture: [Graph and Tree Representations](#)

Lecture and Connected Exercises

Lecture: [Graph and Tree Representations](#)

Lab L4b: [Breadth-First and Depth-First Search](#)

Use an unweighted adjacency list to visit a directed graph with a queue or a stack.

Lecture L4c and lab L4d: [Shortest-Path Algorithms](#)

Retain edge weights and find minimum-cost routes through a process network.

Simple Graphs

A simple graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ stores a set of vertices $\mathcal{V}$ and a set of edges $\mathcal{E}$.

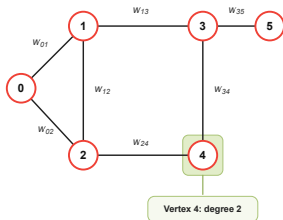
- In an **undirected graph**, an edge $\{u, v\}$ is an unordered pair of distinct vertices and may be followed either way.
- In a **directed graph**, an edge (u, v) is an ordered pair. It points from u to v , and (v, u) is a different edge.
- The word **simple** rules out self-loops and repeated parallel edges.
- An edge may carry a weight such as distance, capacity, time, or cost.

Modeling decision: direction records which transitions are feasible; weight records what one feasible transition costs.

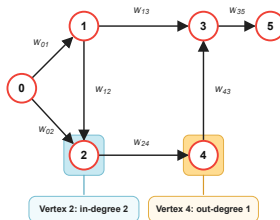
Direction Changes Which Routes Are Feasible

Six vertices and seven edges: $|V| = 6$, $|E| = 7$

Undirected graph



Directed acyclic graph



The undirected drawing allows travel across an edge in either direction. The directed drawing allows only the arrow direction, even when the endpoints match.

Walks, Paths, and Cycles

Let $v_0, v_1, \dots, v_k$ be a sequence of vertices.

- A **walk** follows an edge between every consecutive pair. A directed walk must follow every source-to-target direction.
- A **path** is a walk with no repeated vertices.
- A **cycle** is a closed walk that repeats no edge and repeats no vertex except $v_0 = v_k$.

Walks describe all allowed movement. Paths remove detours. Cycles describe a way to leave a vertex and return to it.

Connectivity Depends on Direction

Connectivity asks whether paths join the vertices.

- An undirected graph is **connected** when a path joins every pair of vertices.
- A directed graph is **weakly connected** when ignoring edge directions produces a connected undirected graph.
- A directed graph is **strongly connected** when every ordered pair (u, v) is joined by a directed path from u to v .

Strong connectivity implies weak connectivity. The reverse implication does not hold.

Directed Degree Has Two Parts

Compare vertex v_4 in the undirected and directed versions of the same graph.

Undirected graph

Two edges touch v_4 :

$$\{v_2, v_4\}, \quad \{v_3, v_4\}.$$

$$\deg(v_4) = 2.$$

Directed graph

One edge enters v_4 and one edge leaves it:

$$v_2 \longrightarrow v_4 \longrightarrow v_3.$$

$$\deg^{\text{in}}(v_4) = 1, \quad \deg^{\text{out}}(v_4) = 1,$$

$$\text{so } \deg(v_4) = 1 + 1 = 2.$$

Key comparison: direction partitions the incident-edge count into incoming and outgoing parts.

Degree Counts Recover the Edge Count

Let $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$. In an undirected graph, summing degrees counts both endpoints of every edge:

$$\underbrace{\sum_{v_i \in \mathcal{V}} \deg(v_i)}_{\text{all incident ends}} = 2m.$$

In a directed graph, each edge contributes once to each directional sum:

$$\underbrace{\sum_{v_i \in \mathcal{V}} \deg^{\text{in}}(v_i)}_{\text{all targets}} = \underbrace{\sum_{v_i \in \mathcal{V}} \deg^{\text{out}}(v_i)}_{\text{all sources}} = m.$$

These **handshaking identities** are useful consistency checks for graph data.

Average Degree Lies Between the Extremes

For an undirected graph with $n \geq 1$, let $\delta(\mathcal{G})$ and $\Delta(\mathcal{G})$ be its minimum and maximum degrees. The average degree is given by:

$$\underbrace{\bar{d}(\mathcal{G}) = \frac{1}{n} \sum_{v_i \in \mathcal{V}} \deg(v_i)}_{\text{average degree}} = \frac{2m}{n}.$$

It is bounded by:

$$\delta(\mathcal{G}) \leq \bar{d}(\mathcal{G}) \leq \Delta(\mathcal{G}).$$

An undirected graph is **regular of degree r** when every vertex has degree r . Then $\delta = \bar{d} = \Delta = r$.

Density Measures the Fraction of Possible Edges

A simple loop-free graph on $n \geq 2$ vertices has maximum edge count:

$$|\mathcal{E}|_{\max} = \begin{cases} n(n-1)/2, & \text{undirected,} \\ n(n-1), & \text{directed.} \end{cases}$$

The density is the observed fraction of that maximum:

$$\underbrace{\rho(\mathcal{G}) = \frac{|\mathcal{E}|}{|\mathcal{E}|_{\max}}}_{\text{graph density}}, \quad 0 \leq \rho(\mathcal{G}) \leq 1.$$

Density near one is **dense**; density near zero is **sparse**. This lecture sets $\rho(\mathcal{G}) = 0$ when $n < 2$.

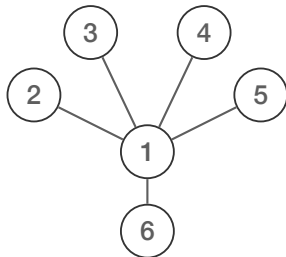
Counts Do Not Determine the Arrangement

A path and a star both have n vertices and $n - 1$ edges, but their distances and degree distributions are very different.



Path graph

$$\Delta = 2, \quad \text{diam} = n - 1.$$



Star graph

$$\Delta = n - 1, \quad \text{diam} = 2.$$

Four Measures Describe Graph Arrangement

For an undirected graph, counts can be supplemented by:

- **Diameter** $\text{diam}(\mathcal{G})$: the largest shortest-path distance in a connected graph.
- **Clique number** $\omega(\mathcal{G})$: the largest number of vertices that are all pairwise adjacent.
- **Chromatic number** $\chi(\mathcal{G})$: the fewest colors that place different colors on adjacent vertices.
- **Independence number** $\alpha(\mathcal{G})$: the largest number of vertices with no edge between any selected pair.

Computational consequence: these measures require edge lookups and neighbor iteration; vertex and edge counts alone are insufficient.

The graph is a mathematical object. Its representation determines what an algorithm can access efficiently.

Three Forms Store the Same Graph

Let $n = |\mathcal{V}|$ and $m = |\mathcal{E}|$.

- An **edge list** stores one source-target-weight record per edge.
- An **adjacency matrix** stores one entry for every ordered pair of vertices.
- An **adjacency list** stores one outgoing-neighbor collection per vertex.

The forms are mathematically equivalent. Their memory layout changes the cost of finding one edge and the cost of iterating over one vertex's neighbors.

Edge Lists Store One Record per Edge

An edge list stores (source, target, weight) for every edge.

- Storage grows as $\Theta(m)$.
- Finding one specified edge takes $\Theta(m)$ time in the worst case without an index.
- Collecting every edge leaving one source also takes $\Theta(m)$ time because the scan may inspect every record.

Edge lists are simple to read, write, and stream. Bellman-Ford also uses them naturally because one pass deliberately scans every edge.

Adjacency Matrices Reserve Every Vertex Pair

For vertex order $v_1, \dots, v_n$, the entry a_{ij} describes (v_i, v_j) :

$$a_{ij} = \begin{cases} 1, & \text{an unweighted edge exists,} \\ w_{ij}, & \text{a weighted edge exists,} \\ 0, & \text{the edge is absent.} \end{cases}$$

- Storage is $\Theta(n^2)$ and one edge lookup is $\Theta(1)$.
- Finding every neighbor of v_i scans row i in $\Theta(n)$ time.
- An undirected matrix is symmetric; a directed matrix need not be.

Weighted graphs: if zero is a valid weight, use a separate marker to distinguish a zero-weight edge from a missing edge.

Adjacency Lists Store Neighbors

An adjacency list stores one collection of outgoing neighbors for each source.

- Unweighted form: `source => [target, ...]`.
- Weighted form: `source => [(target, weight), ...]`.
- Storage is $\Theta(n + m)$ for a directed graph.
- Accessing a source's dictionary entry is expected $\Theta(1)$; iterating over its out-neighbors costs $\Theta(\deg^{\text{out}}(v_i))$.

In an undirected list, each edge appears in both endpoint lists. The exact count is $n + 2m$ stored items, which remains $\Theta(n + m)$.

Pair Lists with an Edge-Weight Map

A flat dictionary can map a known endpoint pair directly to a weight:

$$(u, v) \mapsto w(u, v).$$

- Expected lookup for a known (u, v) is $\Theta(1)$.
- Finding all neighbors of u still requires scanning m entries.
- Pairing the map with an unweighted adjacency list gives fast direct weight lookup and fast neighbor iteration.

Design principle: one graph object may maintain more than one representation when its algorithms require different access patterns.

Storage and Access Costs

Representation	Storage	Find (u, v)	List neighbors of u
Edge list	$\Theta(m)$	$\Theta(m)$	$\Theta(m)$
Adjacency matrix	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n)$
Adjacency list	$\Theta(n + m)$	$\Theta(\deg^{\text{out}}(u))$	$\Theta(\deg^{\text{out}}(u))$

The adjacency-list edge test assumes a linear scan of one neighbor vector. A separate hash-based weight map changes that particular lookup to expected $\Theta(1)$.

Choose the Representation from the Operation

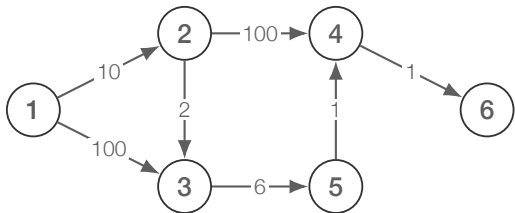
- Use an **adjacency matrix** for a dense graph when the algorithm repeatedly asks whether one specified edge exists.
- Use an **adjacency list** for a sparse graph when the algorithm repeatedly traverses the neighbors of reached vertices.
- Use an **edge list** when the workflow reads, writes, sorts, or scans every edge as a collection of records.

Representation choice is not only about memory. It determines which inner loop the algorithm pays for.

Worked Example: A Six-Vertex Directed Graph

The file `SimpleGraph.txt` contains seven weighted edge records:

source	target	weight
1	2	10
1	3	100
2	3	2
2	4	100
3	5	6
4	6	1
5	4	1



Notebook functions: `read_weighted_edges`, `adjacency_list`, and `adjacency_matrix`.

The Same Edges Become a List or a Matrix

Outgoing adjacency list

1 $\rightarrow$ [2, 3]

2 $\rightarrow$ [3, 4]

3 $\rightarrow$ [5]

4 $\rightarrow$ [6]

5 $\rightarrow$ [4]

6 $\rightarrow$ []

Weighted adjacency matrix

	1	2	3	4	5	6
1	0	10	100	0	0	0
2	0	0	2	100	0	0
3	0	0	0	0	6	0
4	0	0	0	0	0	1
5	0	0	0	1	0	0
6	0	0	0	0	0	0

This lecture's helper stores targets only; the original edge records retain the weights.

Vertex Order Maps Identifiers to Rows

The matrix carries an explicit vertex ordering:

$\text{vertex_order}[i] = \text{vertex represented by row and column } i.$

In the worked example, the order is $[1, 2, 3, 4, 5, 6]$.

- Vertex 1 points to 2 and 3, so the list contains $[2, 3]$.
- Row 1 stores weights 10 and 100 in columns 2 and 3.
- Vertex 6 has an empty neighbor vector because no edge leaves it.

Vertex identifiers need not equal $1 : n$; the ordering provides the translation.

Small Graph, Different Storage Counts

For the six-vertex, seven-edge directed graph:

$$n = 6, \quad m = 7, \quad \rho = \frac{7}{6(6-1)} = \frac{7}{30}.$$

The two storage counts are:

$$\underbrace{n^2 = 36}_{\text{matrix entries}}, \quad \underbrace{n + m = 13}_{\text{directed adjacency-list items}}.$$

Tests in the notebook verify the vertex count, edge count, density, neighbor list for vertex 1, matrix weight for (1, 2), and both storage counts.

Sparse-Graph Storage Diverges at Scale

Consider $n = 100,000$ vertices with mean out-degree 10, so $m = 1,000,000$. Count 8 bytes per matrix entry or list item and ignore container overhead.

$$\underbrace{n^2(8 \text{ bytes})}_{\text{adjacency matrix}} = 10^{10}(8 \text{ bytes}) = \text{80 GB},$$

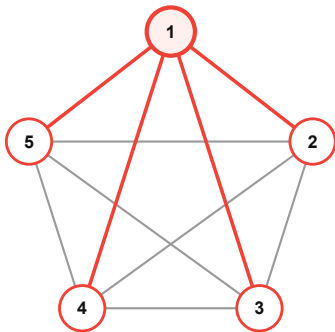
$$\underbrace{(n + m)(8 \text{ bytes})}_{\text{adjacency list}} = 1.1 \times 10^6(8 \text{ bytes}) = \text{8.8 MB}.$$

The matrix grows with n^2 ; the list grows with the vertices and the edges that actually exist.

Three graph families turn structural definitions into
recognition tests, formulas, and specialized algorithms.

Complete Graphs Use Every Possible Edge

A complete graph K_n is a simple undirected graph with an edge between every pair of distinct vertices.



What the picture shows

Degree of every vertex

$$\deg(v) = 5 - 1 = 4$$

Number of edges

$$|E| = 5(4)/2 = 10$$

Fraction of possible edges

$$\rho(K_5) = 1$$

— edges incident to vertex 1

Complete-Graph Measures Follow Directly

For $n \geq 2$, every possible edge is present:

$$|\mathcal{E}| = \binom{n}{2} = \frac{n(n-1)}{2}, \quad \deg(v_i) = n-1.$$

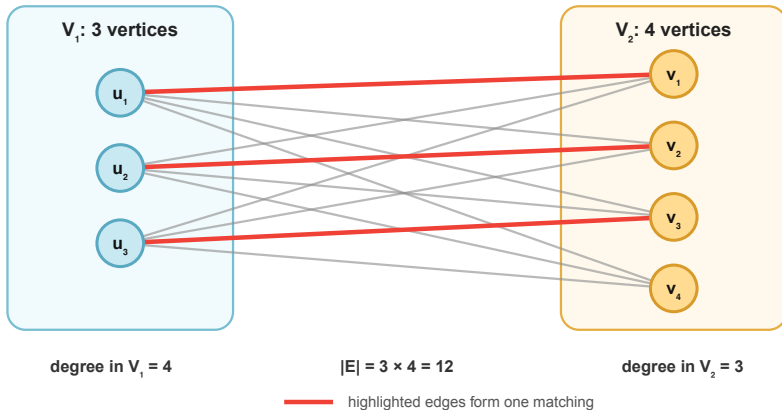
Consequently:

$$\begin{aligned} \rho(K_n) &= 1, & \text{diam}(K_n) &= 1, \\ \omega(K_n) &= \chi(K_n) = n, & \alpha(K_n) &= 1. \end{aligned}$$

Round-robin schedules have the edge pattern of K_n . Traveling-salesman models use a weighted K_n when every city pair has a travel cost.

Bipartite Graphs Split Vertices into Two Groups

A graph is bipartite when $\mathcal{V} = \mathcal{V}_1 \dot{\cup} \mathcal{V}_2$ and every edge has one endpoint in each group.



Three Equivalent Tests Identify Bipartite Graphs

For an undirected graph $\mathcal{G}$, the following statements are equivalent:

1. $\mathcal{G}$ is bipartite.
2. $\mathcal{G}$ can be colored with at most two colors, so $\chi(\mathcal{G}) \leq 2$.
3. $\mathcal{G}$ contains no odd-length cycle.

A traversal tests this in $\Theta(|\mathcal{V}| + |\mathcal{E}|)$ time: color each uncolored neighbor with the opposite color and reject any same-color edge.

Matchings Assign Distinct Options

A **matching** is a set of edges that share no vertices. A matching that covers $\mathcal{V}_1$ assigns a different right-side option to every left-side vertex.

For $S \subseteq \mathcal{V}_1$, let $N(S) \subseteq \mathcal{V}_2$ be all neighbors of vertices in S . Hall's theorem states:

$$\underbrace{|N(S)| \geq |S|}_{\text{a matching covers } \mathcal{V}_1} \quad \text{for every } S \subseteq \mathcal{V}_1.$$

Every group on the left must collectively have at least as many available options on the right. A **perfect matching** covers both groups and therefore requires $|\mathcal{V}_1| = |\mathcal{V}_2|$.

Bipartite Matrices Have Block Structure

List the vertices of $\mathcal{V}_1$ before the vertices of $\mathcal{V}_2$. The adjacency matrix becomes:

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{0} \end{bmatrix}.$$

The $m \times n$ **biadjacency matrix** $\mathbf{B}$ stores all edges between the groups. Both diagonal blocks are zero because no edge stays within one group.

For an undirected bipartite graph, storing $\mathbf{B}$ determines $\mathbf{B}^\top$ and both zero blocks.

Trees Are Minimally Connected Graphs

A tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$ is a connected undirected graph with no cycles.

- Exactly one path joins every pair of vertices.
- Removing any edge disconnects the tree.
- Adding an edge between nonadjacent vertices creates exactly one cycle.

A tree has exactly enough edges to remain connected and no edge that can be removed without losing connectivity.

Six Equivalent Tests Characterize a Tree

For an undirected graph with $n \geq 1$, each statement is equivalent:

1. The graph is a tree.
2. It is connected and has $n - 1$ edges.
3. It has no cycle and has $n - 1$ edges.
4. It is connected, and removing any edge disconnects it.
5. It has no cycle, and adding any missing edge creates one cycle.
6. Every pair of vertices is joined by exactly one path.

Use: choose the test whose information is cheapest to verify in the problem at hand.

Trees Stay Connected While Becoming Sparse

Every tree on n vertices has $n - 1$ edges. For $n \geq 2$, its density is:

$$\rho(\mathcal{T}) = \frac{n - 1}{n(n - 1)/2} = \underbrace{\frac{2}{n}}_{\text{tree density}}.$$

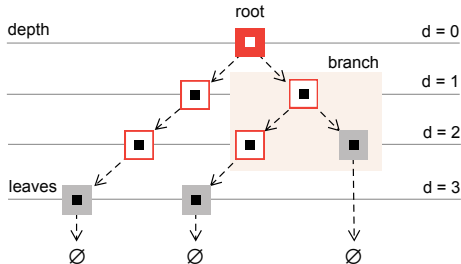
Thus $\rho(\mathcal{T}) \rightarrow 0$ as $n \rightarrow \infty$, even though the tree remains connected.

Sparsity does not mean fragmentation. A tree is the sparsest possible connected undirected graph.

A Root Gives a Tree Parent-Child Structure

Trees organize nodes with parent-child links.

Trees can represent a file system, organization chart, or manufacturing process.



The root has no parent. Leaves have no children.

The tree's height is the number of edges in its longest root-to-leaf path.

Every non-root vertex has one parent. Vertices may have children; those with no children are leaves. Depth counts edges from the root, and height is the maximum depth.

Spanning Trees Retain Connectivity

Every connected undirected graph $\mathcal{G}$ contains a spanning tree $\mathcal{T}$ with all n vertices and exactly $n - 1$ edges.

- A cycle-free graph that is not connected is a **forest**; each connected component is a tree.
- Rooted trees model file systems, organization charts, and function calls.
- Specialized tree structure can make difficult graph problems easier. A largest independent set is found by tree dynamic programming in $\Theta(n)$ time, while the general-graph problem is NP-hard.

Traversal Reads Only Reached Neighbor Lists

A traversal starts at one vertex, reads its neighbors, and repeats until no unvisited reachable vertex remains.

- Breadth-first search uses a queue and visits vertices in layers.
- Depth-first search follows one branch before backtracking.
- Edge weights do not affect either visit order, so lab L4b uses the unweighted adjacency list.

Both run in $\Theta(|\mathcal{V}_r| + |\mathcal{E}_r|)$ for the vertices and outgoing edges reached from the start.

The Algorithm Selects the Representation

Dijkstra

Groups weighted edges by source:

$$u \mapsto [(v, w(u, v)), \dots].$$

Each iteration needs the target and weight of every outgoing edge.

The mathematical graph is the same. The global update schedule determines the memory layout that is convenient.

Bellman-Ford

Uses an edge list:

$$[(u, v, w(u, v)), \dots].$$

Each relaxation pass deliberately scans every edge.

Summary

A graph states which vertex pairs are joined. Its storage form sets the cost of looking up one edge and listing one vertex's neighbors.

Key takeaways

- **Count vertices and edges.** Degree and density summarize incidence and fullness, but not arrangement.
- **Recognize structural families.** Complete graphs, bipartite graphs, and trees each have equivalent tests and characteristic counts.
- **Match storage to the operation.** Matrices favor direct edge lookup; adjacency lists favor sparse traversal; edge lists favor full scans.

L4b turns an adjacency list into a systematic visit order; L4c restores weights and minimizes route cost.

That's a wrap. Which graph representation would you choose for the next algorithm, and what operation justifies that choice?